

# Divide and Conquer – Complexity Notation

# Motivation

Consider the Binary Search function we wrote in class

---

*BinarySearch*(*A*, *x*, *min*, *max*)

---

**Input:** An array of  $n$  sorted numbers  $A$ , number  $x$ , integer indices  $lo$ , and  $hi$ . (Initially  $lo = 1$  and  $hi = n$ )

**Output:** An index of  $x$  in  $A$  if  $x \in A$ , or  $-1$  if  $x \notin A$ .

```
1: if  $hi < lo$  then  
    return  $-1$   
2: else  
3:    $mid \leftarrow \lfloor (hi+lo)/2 \rfloor$   
4:   if  $x < A[mid]$  then  
       return BinarySearch(A, x, lo,  $mid - 1$ )  
5:   else if  $x > A[mid]$  then  
       return BinarySearch(A, x,  $mid + 1$ , hi)  
6:   else  
       return mid
```

---

How much time (or how many instructions) does it take to run?

# Motivation

---

A precise answer faces several complications:

- ❑ Query-dependent:
  - ❑ The same list may take a different amount of time depending on the query
  - ❑ For each recursive call, the exact execution path depends on the query
- ❑ Architecture-dependent:
  - ❑ Does division take the same number of CPU cycles as addition?
  - ❑ If we removed line 3 (definition of mid) and calculated the index twice, would that:
    - ❑ Make the program faster (one fewer variable attribution)?
    - ❑ Make the program slower (redundancy in calculating the mid index twice)

# Motivation

---

A more practical approach is to take a less granular view

- ❑ Analyze only the worst case (or average case) instead of every possible case
  - ❑ For *BinarySearch* this is when  $x \notin A$
- ❑ Consider only how time depends on the input *size* rather than every instruction
  - ❑ If we double the size of the array, we need about one more
- ❑ If two scenarios differ only by a constant multiplicative factor, we consider that they cost the same
  - ❑ This way, we don't worry whether one recursive call takes 2, 5, 6, 10 etc. steps
  - ❑ We can just say "the maximum number of steps (in one call) does not depend on the size of the array".
    - ❑ That is, it "constant" time

# Big O notation

One way to formalize this is with *Big O notation*:

## Definition:

Consider two functions  $f(n), g(n)$  of the natural numbers

We say  $f(n) = O(g(n))$  if there are constants  $c > 0, n_0 > 0$  such that  $\forall n > n_0 \ f(n) \leq c \cdot g(n)$

We read  $f(n) = O(g(n))$  as “ $f$  is O of  $g$ ” or “ $f$  is big O of  $g$ ”

In other words,

- if  $n$  is large enough,  $f(n) \leq g(n)$  up to some multiplicative constant  $c$
- Alternatively,  $g$  is an upper bound of  $f$  for large enough  $n$

Interpretation

- We typically interpret  $f$  as the runtime of our program as a function of input size
- We typically take  $g(n)$  as one of several common complexity classes
  - Examples:  $\log n, n, n \cdot \log n, n^2, 2^n, n!$
- Therefore we might say something like  $f = O(n^2)$

# Big O application

Let's show how this helps with an example  
Consider the function

$$f(n) = 3n^2 + 5n + 7$$

We will show that  $f = O(n^2)$

- Take  $n_0 = 1$ . For all  $n \geq 1$ :
  - $7 \leq 7n^2$
  - $5n \leq n^2$
- Therefore,
  - $f(n) \leq 3n^2 + 5n^2 + 7n^2 = 15n^2$
  - In other words, we have  $f(n) \leq 15n^2$  whenever  $n \geq 1$  so  $f = O(n^2)$
- A tighter bound: if  $n \geq 7$ , then  $f(n) \leq 4n^2$  (shown in class)
  - This corresponds to  $n_0 = 7, c = 4$  in the definition

# Divide and Conquer

---

Divide and Conquer is a strategy that solves a problem by:

- 1) Breaking the problem into subproblems that are themselves smaller instances of the same type of problem.
- 2) Recursively solving these subproblems.
- 3) Appropriately combining their answers.

# Searching

## Search

**Input:** A list of numbers, sorted in increasing order,  $A = [a_0, a_1, a_2, a_3, \dots, a_{n-1}]$  and a number  $x$ .

**Goal:** Return a position of  $x$  in  $A$  if  $x \in A$  or return a statement that  $x$  is not in the list.

- ☐ How would a four-year-old do this?
- ☐ How would a 2nd grader do this?
- ☐ How would you do this?

# BinarySearch

---

We must ask ourselves two questions about this algorithm:

- ❑ Is it correct?
- ❑ What is its running time?

At each stage we divide the problem in half and it takes constant time to “combine” solutions:

$$T(n) = T(n/2) + O(1)$$

Later, we will learn how to “solve” this recurrence relationship into something like  $O(\log n)$

# BinarySearch - Correctness

---

We must ask ourselves two questions about this algorithm:

- ❑ Is it correct?
- ❑ What does "correct" mean?
  - ❑ Look at the problem description
- ❑ The program must:
  - ❑ Always halt (rather than looping forever)
  - ❑ Return  $-1$  if  $x$  does not exist in the search range
  - ❑ Return an index of  $x$  if it does exist in the search range

# BinarySearch - Correctness

While we could prove all requirements for correctness in one go, let's break it down

## Lemma 1

*BinarySearch always halts*

Sketch of proof

1. On every iteration, either  $lo$  increases by at least 1, or  $hi$  decreases by at least 1
2. Therefore, the quantity  $n = hi - lo + 1$  is strictly decreasing
3. Therefore, at some point we must reach the base case  $hi < lo$

## Theorem 1

*If  $x$  is not an element of the list, BinarySearch will return  $-1$ .*

Proof (by contradiction)

1. Lemma 1 tells us that *BinarySearch* always halts, so it must return some value  $r$
2. Suppose  $x \notin A$ , that is  $\forall i, A[i] \neq x$  and yet  $r \neq -1$
3. The only step where the algorithm terminates on a value other than  $-1$  is step 6
4. But this only happens if it is neither the case that  $x < A[mid]$  nor  $x > A[mid]$
5. Therefore,  $x = A[mid]$ . Contradiction with line 2 of the proof

# BinarySearch - Correctness

While we could prove all requirements for correctness in one go, let's break it down

## Theorem 2

*If  $x \in A$ , Binary Search returns an index of  $x$  in  $A$*

Proof by induction on  $n = hi - lo + 1$ , shown in class